The Perceptron Algorithm

CS 115B: Fundamentals of NLP — Neural Networks I

James Pustejovsky

Brandeis University

Spring 2026

Today's Lecture

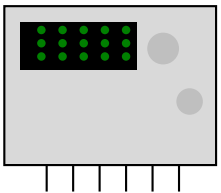
1. Historical Context
2. Linear Classifiers
3. The Perceptron Algorithm
4. Worked Examples
5. Convergence and Limitations
6. Limitations and the Path Forward
7. Summary

The Birth of the Perceptron

Frank Rosenblatt (1928–1971)

- Psychologist at Cornell Aeronautical Laboratory
- 1957: Proposed the Perceptron
- 1958: Published "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain"
- Built the **Mark I Perceptron** — hardware implementation

Goal: A machine that could **learn** to recognize patterns, inspired by biological neurons.



Mark I Perceptron (1958)

400 photocells, weights adjusted by electric motors

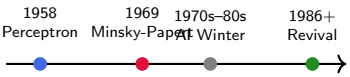
The Hype and the Fall

The New York Times, July 8, 1958:

> *"The Navy revealed the embryo of an electronic computer today that it expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence."*

Then came Minsky & Papert (1969):

- Proved perceptrons can't compute XOR
- Led to the first "AI Winter"
- But their critique applies to *any* linear classifier



Lesson: The perceptron is limited, but understanding *why* leads to neural networks.

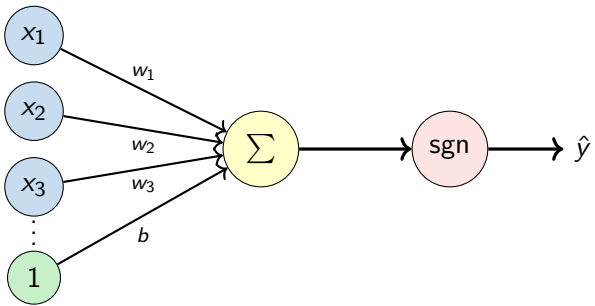
From Biology to Mathematics

Biological neuron:

- Multiple input signals
- Weighted by synapse strength
- Summed in cell body
- Fires if above threshold

Mathematical model:

- Input features $x_1, x_2, \ldots, x_d$
- Weights $w_1, w_2, \ldots, w_d$
- Weighted sum: $\sum_i w_i x_i$
- Output: $\text{sign}(\sum_i w_i x_i + b)$



$$\hat{y} = \text{sign}\left(\sum_{i=1}^{d} w_i x_i + b\right)$$

Linear Classifiers: Setup

Binary classification problem:

- Inputs: d-dimensional vectors $\mathbf{x} \in \mathbb{R}^d$
- Outputs: Labels $y \in \{-1, +1\}$

Linear Threshold Unit (LTU):
Parameters: weight vector $\mathbf{w} \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) = \text{sign}\left(\sum_{i=1}^{d} w_i x_i + b\right)$$

Classification rule:

$$\mathbf{w}^\top \mathbf{x} + b \geq 0 \implies \hat{y} = +1 \qquad \mathbf{w}^\top \mathbf{x} + b < 0 \implies \hat{y} = -1$$

Note: We use $\{-1, +1\}$ labels (not $\{0, 1\}$) for mathematical convenience.

Geometry of Linear Classifiers

The decision boundary:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

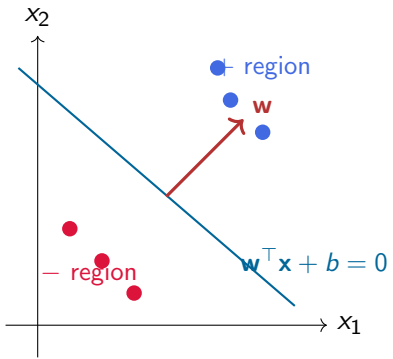
This defines a **hyperplane** that separates the space into two half-spaces.

Key geometric facts:

- **w** is **perpendicular** to the decision boundary
- **w** points toward the positive region
- Scaling **w** doesn't change the boundary

The Bias Trick: Append 1 to **x**, fold b into **w**:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$$



The Perceptron Algorithm

Input: Training examples $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n)$
where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$

1. Initialize $\mathbf{w} = \mathbf{0}$
2. For each training example $(\mathbf{x}_i, y_i)$:
   1. Predict: $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x}_i)$
   2. If $\hat{y} \neq y_i$ (mistake!):

$$\mathbf{w} \leftarrow \mathbf{w} + y_i \cdot \mathbf{x}_i$$

3. Repeat until convergence (or for T epochs)

Key property: Update **only on errors** — this is *online*, *mistake-driven* learning

Understanding the Update Rule

$$\mathbf{w} \leftarrow \mathbf{w} + y_i \cdot \mathbf{x}_i$$

Two cases:

False negative ($y_i = +1$, predicted -1):

$$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}_i$$

- Add $\mathbf{x}_i$ to $\mathbf{w}$
- **Increases** $\mathbf{w}^\top \mathbf{x}_i$
- Pushes score toward positive

False positive ($y_i = -1$, predicted $+1$):

$$\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}_i$$

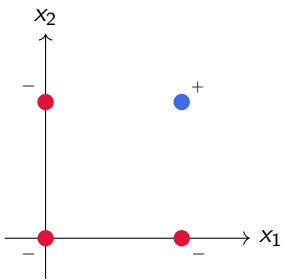
- Subtract $\mathbf{x}_i$ from $\mathbf{w}$
- **Decreases** $\mathbf{w}^\top \mathbf{x}_i$
- Pushes score toward negative

Unified condition: A mistake occurs iff $y_i \cdot (\mathbf{w}^\top \mathbf{x}_i) \leq 0$

Example 1: Learning AND

Training data for AND function:

x_1	x_2	y
0	0	-1
0	1	-1
1	0	-1
1	1	$+1$



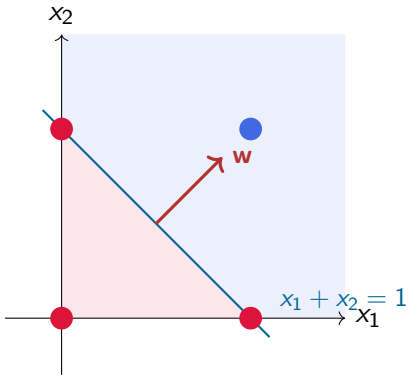
Initialize: $\mathbf{w} = (w_1, w_2, b) = (0, 0, 0)$ (with bias trick: $\mathbf{x} \to (x_1, x_2, 1)$)

Example 1: Trace Through

Step	$\mathbf{x}$	y	$\mathbf{w}^\top\mathbf{x}$	$\hat{y}$	Update
1	$(0,0,1)$	-1	0	$+1$	$\mathbf{w} = (0,0,0) + (-1)(0,0,1) = (0,0,-1)$
2	$(0,1,1)$	-1	0	$+1$	$\mathbf{w} = (0,0,-1) + (-1)(0,1,1) = (0,-1,-2)$
3	$(1,0,1)$	-1	-1	-1	No update (correct!)
4	$(1,1,1)$	$+1$	-2	-1	$\mathbf{w} = (0,-1,-2) + (+1)(1,1,1) = (1,0,-1)$
5	$(0,0,1)$	-1	-1	-1	No update
6	$(0,1,1)$	-1	-1	-1	No update
7	$(1,0,1)$	-1	0	$+1$	$\mathbf{w} = (1,0,-1) + (-1)(1,0,1) = (0,0,-2)$
8	$(1,1,1)$	$+1$	-1	-1	$\mathbf{w} = (0,0,-2) + (+1)(1,1,1) = (1,1,-1)$
9–12	All four examples correct!				**Converged!**

Final: $\mathbf{w} = (1,1,-1) \Rightarrow \hat{y} = \text{sign}(x_1 + x_2 - 1)$

Example 1: Visualization



The perceptron successfully learned AND in 8 iterations!

Example 2: Sentiment Classification

Task: Classify movie reviews as positive ($+1$) or negative (-1)

Features: Word counts (bag of words)

Review	great	terrible	loved	boring	waste	y
"Great movie, loved it"	1	0	1	0	0	$+1$
"Terrible, boring waste"	0	1	0	1	1	-1
"Loved the great acting"	1	0	1	0	0	$+1$
"Boring and terrible"	0	1	0	1	0	-1

Initialize: $\mathbf{w} = (0, 0, 0, 0, 0, 0)$ (5 words + bias)

Example 2: Trace Through

Iteration 1:

- Review 1: $\mathbf{x} = (1, 0, 1, 0, 0, 1)$, $y = +1$, score $= 0$, predict $+1$
- Correct! (We use $\text{sign}(0) = +1$)

Iteration 2:

- Review 2: $\mathbf{x} = (0, 1, 0, 1, 1, 1)$, $y = -1$, score $= 0$, predict $+1$
- Mistake! $\mathbf{w} \leftarrow (0, 0, 0, 0, 0, 0) + (-1)(0, 1, 0, 1, 1, 1) = (0, -1, 0, -1, -1, -1)$

Interpretation of learned weights:

Word	Weight	Meaning
great	0	(not yet learned)
terrible	-1	negative indicator
loved	0	(not yet learned)
boring	-1	negative indicator
waste	-1	negative indicator

Example 2: After Full Training

After processing all examples multiple times:

Final weights:

Word	Weight	Interpretation
great	$+2$	strong positive signal
terrible	-2	strong negative signal
loved	$+2$	strong positive signal
boring	-2	strong negative signal
waste	-1	negative signal
bias	0	neutral baseline

New review: "The movie was great but boring"
$\mathbf{x} = (1, 0, 0, 1, 0, 1)$
Score $= 1 \cdot 2 + 0 + 0 + 1 \cdot (-2) + 0 + 0 = 0$
Borderline case! The positive "great" and negative "boring" cancel out.

Example 3: Spam Detection

Features for email classification:

- x_1: Contains "free"
- x_2: Contains "winner"
- x_3: Contains "meeting"
- x_4: Has attachment
- x_5: From known contact

Learned weights might be:

$w_1 = +3$ ("free" $\rightarrow$ spam)
$w_2 = +4$ ("winner" $\rightarrow$ spam)
$w_3 = -2$ ("meeting" $\rightarrow$ not spam)
$w_4 = +1$ (attachment $\rightarrow$ slight spam)
$w_5 = -5$ (known contact $\rightarrow$ not spam)

Test email:
"Free meeting tomorrow"
From: known contact

$\mathbf{x} = (1, 0, 1, 0, 1, 1)$

$\text{Score} = 3 + 0 - 2 + 0 - 5 + b$
$= -4 + b$

If $b = 0$: Score < 0
$\Rightarrow$ **Not spam** ✓

Known contact overrides "free"

Does the Perceptron Always Converge?

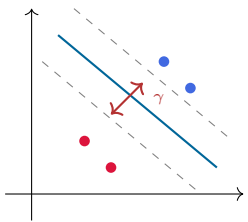
Perceptron Convergence Theorem (Novikoff, 1962)

If the training data is **linearly separable**, the perceptron algorithm will find a separating hyperplane in a finite number of updates.

More precisely: If data has margin γ and radius R:

$$\text{Number of mistakes} \leq \left(\frac{R}{\gamma}\right)^2$$

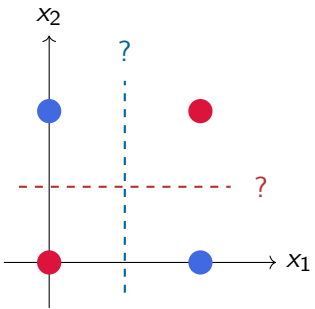
- $R = \max_i \|\mathbf{x}_i\|$ (data spread)
- $\gamma =$ margin (separation quality)
- Larger margin $\Rightarrow$ faster convergence



What If Data Is NOT Linearly Separable?

The XOR Problem:

x_1	x_2	y
0	0	-1
0	1	$+1$
1	0	$+1$
1	1	-1



No single line can separate the classes!

The perceptron will **never converge** — it will keep updating forever, cycling through the data.

Practical Variants

Problem: Even when data is separable, which separator do we get?

Averaged Perceptron (Collins, 2002):

- Keep running sum of all weight vectors
- Return the average at the end
- Much better generalization
- **Use this in practice!**

$$\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{w}_t$$

Margin Perceptron:

- Update even if "barely correct"
- Update when $y_i \cdot (\mathbf{w}^\top \mathbf{x}_i) < \tau$
- Encourages larger margins
- Related to SVMs

What's Wrong with the Perceptron?

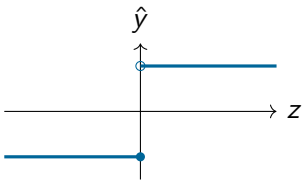
1. **Only linear boundaries**
   - Can't solve XOR or any non-linearly-separable problem
   - Solution: Multi-layer networks (later in course)
2. **No probabilities**
   - Output is just $+1$ or -1
   - Can't say "80% confident this is spam"
   - **Solution: Logistic Regression (next!)**
3. **Treats all correct predictions equally**
   - No update whether margin is 0.001 or 1000
   - Solution: Margin-based methods (SVMs)
4. **No principled loss function**
   - Hard to compare to other methods
   - Hard to regularize
   - **Solution: Logistic Regression**

From Perceptron to Logistic Regression

Key insight: Replace the hard sign function with a soft **sigmoid**

Perceptron:

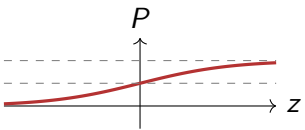
$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$$



Hard threshold

Logistic Regression:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$$



Smooth probability

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{(sigmoid function)}$$

Why Logistic Regression?

Perceptron gives:

- Binary prediction only
- "This email IS spam"
- No confidence measure
- Can't rank predictions

Logistic Regression gives:

- Probability of each class
- "95% chance this is spam"
- Can threshold at any level
- Can rank by confidence

Example outputs for different emails:

Email	Perceptron	Logistic Regression
"FREE WINNER!!!"	Spam	$P(\text{spam}) = 0.99$
"Meeting tomorrow"	Not spam	$P(\text{spam}) = 0.02$
"Free lunch at meeting"	Spam	$P(\text{spam}) = 0.45$

The third email is **uncertain** — logistic regression tells us this!

The Logistic Regression Model

For binary classification ($y \in \{0, 1\}$):

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

$$P(y = 0|\mathbf{x}) = 1 - \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{e^{-\mathbf{w}^\top \mathbf{x}}}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

Decision rule: Predict $y = 1$ if $P(y = 1|\mathbf{x}) > 0.5$

Key properties of sigmoid:

- Output always in $(0, 1)$ — valid probability
- $\sigma(0) = 0.5$ — decision boundary at $\mathbf{w}^\top \mathbf{x} = 0$
- $\sigma(-z) = 1 - \sigma(z)$ — symmetric
- Smooth and differentiable everywhere

How to Learn Weights?

Perceptron: Mistake-driven updates (no explicit objective)

Logistic Regression: Maximize likelihood of training data

$$L(\mathbf{w}) = \prod_{i=1}^{n} P(y_i|\mathbf{x}_i; \mathbf{w})$$

Equivalently, minimize **negative log-likelihood** (cross-entropy loss):

$$\mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{n} \Big[y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i)) \Big]$$

Optimization: Gradient descent

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$$

Next lecture: We'll derive this and see how to optimize it!

Summary

The Perceptron:

- Linear classifier: $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x})$
- Online, mistake-driven learning
- Update: $\mathbf{w} \leftarrow \mathbf{w} + y \cdot \mathbf{x}$ on errors
- Converges for linearly separable data

Practical tips:

- Use averaged perceptron for better generalization
- Multiple passes (epochs) through data
- Works well for high-dimensional text classification

Limitations:

- Only linear boundaries (can't do XOR)
- No probabilities, no principled loss function

Next: Logistic Regression — probabilistic linear classification

Coming Up

Next lecture: Logistic Regression

- Sigmoid function and probabilistic output
- Cross-entropy loss function
- Gradient descent optimization
- Regularization

Then: Multi-Layer Perceptrons

- Hidden layers for non-linear boundaries
- Solving XOR and beyond
- Backpropagation algorithm
- Deep learning foundations

The perceptron is the foundation — now we build on it!